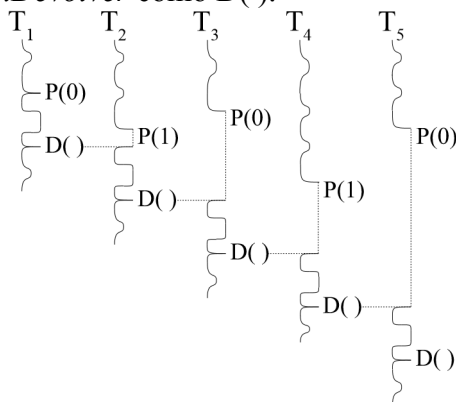


Se dispone de un recurso único compartido entre varios threads. Los threads se agrupan en categoría 0 y categoría 1. Se dice que 0 es la categoría opuesta de 1, y 1 la categoría opuesta de 0. Un thread de categoría *cat* solicita el recurso invocando la función *pedir(cat)* y devuelve el recurso llamando a la función *devolver()*, sin parámetros. Los encabezados para ambas funciones son:

```
int pedir(int cat);
void devolver();
```

Programa las funciones *pedir* y *devolver* garantizando la exclusión mutua al acceder al recurso compartido. Se requiere una política de asignación alternada primero y luego por orden de llegada. Esto significa que cuando un thread de categoría *cat* devuelve el recurso, si hay algún thread en espera de la categoría opuesta a *cat*, el recurso se asigna inmediatamente al thread de categoría opuesta que lleva más tiempo esperando. Si no hay threads en espera de la categoría opuesta pero sí de la misma categoría *cat*, el recurso se asigna al thread que lleva más tiempo en espera. Si no hay ningún thread en espera, el recurso queda disponible y se asignará en el futuro al primer thread que lo solicite, cualquiera sea su categoría. El siguiente diagrama muestra un ejemplo de asignación del recurso. La invocación de *nPedir* se abrevió como P(...) y la de *nDevolver* como D().



Inicialice las variables globales en la función *iniciar* y libere los recursos solicitados en *terminar*.

Requerimientos

- 1) Ud. **debe** programar las funciones solicitadas en el archivo *pedir.c*, usando **un solo mutex** y **una sola condición** de pthreads. No puede usar múltiples condiciones u otras herramientas de sincronización como por ejemplo semáforos.
- 2) Note que debe garantizar el orden de los threads según el tiempo de espera para ambas categorías. Para ello **debe** utilizar dos **distribuidores de números** (uno para cada categoría) como se usó para el problema de los lectores/escritores visto en cátedra. **No puede usar colas fifo** para almacenar los threads en espera.

Instrucciones

Baje *t2.zip* de U-cursos y descomprímalo. El directorio *T2* contiene los archivos *test-pedir.c*, *Makefile*, *pedir.h* (con los encabezados requeridos) y otros archivos. Ud. debe programar en el archivo *disco.c* las funciones solicitadas. Defina otras funciones si las necesita.

Pruebe su tarea bajo Debian 13. Ejecute el comando *make* sin parámetros. Le mostrará las opciones que tiene para compilar su tarea. Estos son los requerimientos para aprobar su tarea:

- *make run* debe felicitarlo por aprobar este modo de ejecución.
- *make run-g* debe felicitarlo.
- *make run-san* debe felicitarlo y no reportar ningún incidente en el manejo de memoria.
- *make run-thr* debe felicitarlo y no reportar ningún datarace.

Cuando pruebe su tarea con *make run* en su computador asegúrese de que esté configurado en modo alto rendimiento y que no estén corriendo otros procesos intensivos en uso de CPU al mismo tiempo. Invoque el comando *make zip* para ejecutar todos los tests y generar un archivo *disco.zip* que contiene *pedir.c*, con su solución, y *resultados.txt*, con la salida de *make run*, *make run-g*, *make run-san* y *make run-thr*.

Entrega

Ud. solo debe entregar por medio de U-cursos el archivo *pedir.zip* generado por *make zip*. Recuerde descargar el archivo que subió, descargar nuevamente los archivos adjuntos y volver a probar la tarea tal cual como la subió a U-cursos. Solo así estará seguro de no haber entregado archivos incorrectos. Se descuenta medio punto por día de atraso. No se consideran los días de receso, sábados, domingos o festivos.